



UNIVERSIDAD SANTO TOMÁS SECCIONAL TUNJA
Facultad de Ingeniería de Sistemas

DESARROLLO SEGURO DE SOFTWARE

Laboratorio No. 9 · Autenticación con JWT · API_TEATRO

ESPACIO ACADÉMICO:
Seguridad en el Desarrollo de Software

PRESENTADO A:
Sergio Arley Puerto Moreno

AUTORES:
Juan Sebastián Bonilla León
Juan Camilo Calderón Delgado
Silvana Sofía Siza Soriano
Cristian Emanuel Hernández Araque
Cristian Rodrigo Amaya Torres

2026

1. El problema: del login a una identidad que viaja

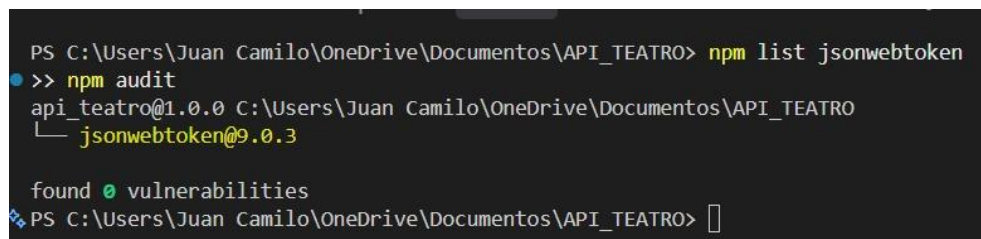
Al terminar el Laboratorio 8 la API ya registraba usuarios con contraseñas protegidas con bcrypt y con un rol que decide el servidor. Pero el login tenía un límite: respondía "Autenticación correcta" y ahí se acababa todo. En la petición siguiente la API ya no sabía quién era esa persona. Para que lo supiera, el cliente tendría que mandar el correo y la contraseña en cada petición, y eso es justo lo que no queremos: la contraseña viajaría todo el tiempo.

La solución de este laboratorio es el JWT (JSON Web Token). Funciona como la manilla que entregan a la entrada de un evento: te identificas una sola vez en la puerta, te ponen la manilla, y de ahí en adelante muestras la manilla, no la cédula. Si alguien intenta falsificarla o ya se venció, no entra.

Aquí la puerta es `POST /api/auth/login`. Si las credenciales son correctas, el servidor firma un token con un secreto que solo él conoce y se lo entrega al cliente. Después el cliente lo manda en la cabecera `Authorization: Bearer <token>` y el servidor solo tiene que comprobar la firma para saber quién es, sin volver a pedir la contraseña.

2. La librería y el secreto

Instalamos `jsonwebtoken`, la librería estándar de Node para firmar y verificar tokens. Quedó en la versión 9.0.3 y `npm audit` siguió en 0 vulnerabilidades:



```
PS C:\Users\Juan Camilo\OneDrive\Documentos\API_TEATRO> npm list jsonwebtoken
api_teatro@1.0.0 C:\Users\Juan Camilo\OneDrive\Documentos\API_TEATRO
└─ jsonwebtoken@9.0.3

found 0 vulnerabilities
PS C:\Users\Juan Camilo\OneDrive\Documentos\API_TEATRO>
```

Figura 1. `jsonwebtoken@9.0.3` instalado en el proyecto y `npm audit` sin vulnerabilidades.

El token se firma con un secreto, `JWT_SECRET`. Quien conozca ese secreto puede fabricar tokens válidos para cualquier usuario, así que se trata igual que las API Keys: vive solo en el `.env`, que está en `.gitignore` y nunca sale en capturas. En `.env.example` agregamos las dos variables nuevas con valores de relleno, para que quien clone el proyecto sepa qué debe configurar:

- `JWT_SECRET`: el secreto de la firma. En el `.env` real pusimos una cadena larga y aleatoria.
- `JWT_EXPIRES_IN`: cuánto dura el token. Lo dejamos en `1h`.

```

.env.example
1  PORT=3000
2  ALLOWED_ORIGIN=http://localhost:3000
3  RATE_LIMIT_MAX=100
4
5  # Claves de los clientes que consumen /api. Solo se guarda su hash en memoria
6  API_KEY_POSTMAN=REEMPLAZAR_CON_API_KEY_SEGURA
7  API_KEY_TAQUILLA=REEMPLAZAR_CON_API_KEY_SEGURA
8  API_KEY_MOVIL=REEMPLAZAR_CON_API_KEY_SEGURA
9
10 # Firma de los JWT que entrega el login
11 JWT_SECRET=REEMPLAZAR_CON_SECRETO_JWT_SEGURO
12 JWT_EXPIRES_IN=1h
13

```

Figura 2. .env.example con las variables `JWT_SECRET` y `JWT_EXPIRES_IN`. Solo lleva valores de relleno; los reales están en el .env.

3. Qué viaja dentro del token

Un JWT tiene tres partes separadas por puntos: cabecera, payload y firma. La cabecera dice con qué algoritmo se firmó, el payload lleva los datos (los claims) y la firma demuestra que nadie los cambió. En nuestro token decidimos llevar lo mínimo:

Claim	Qué guarda	Quién lo pone
email	El correo del usuario	Nosotros, en el payload
rol	El rol asignado por el servidor (asistente)	Nosotros, en el payload
sub	El id del usuario (claim estándar "subject")	jsonwebtoken, con la opción <code>subject</code>
iat	Cuándo se emitió (segundos desde 1970)	jsonwebtoken, automático
exp	Cuándo vence	jsonwebtoken, con la opción <code>expiresIn</code>

Lo que nunca va en el token es la contraseña ni su hash, ni nada que no queramos que el cliente vea.

Firmar no es cifrar

Esta fue la idea que más nos costó al principio. El payload solo está codificado en base64, no cifrado: cualquiera que tenga el token puede leerlo. Lo que hace la firma es impedir que alguien lo modifique sin que el servidor se dé cuenta. Por eso el token sirve para demostrar quién eres, pero no para esconder información. Más adelante, en la sección 9, lo comprobamos leyendo un token sin conocer el secreto.

4. Generar y verificar: jwt.util.js

Creamos `src/utills/jwt.util.js` con dos funciones, para que el resto del proyecto nunca llame directamente a la librería:

- `generarToken(usuario)` firma `{ email, rol }`, pone el id en `subject` y toma la duración de `JWT_EXPIRES_IN` (si no está definida, usa una por defecto).
- `verificarToken(token)` comprueba la firma y la fecha de vencimiento. Si algo falla, lanza un error.

Un detalle importante es el algoritmo. Al firmar fijamos `algorithm: ALGORITMO` (HS256) y al verificar le pasamos `algorithms: [ALGORITMO]`, una lista con un solo valor permitido:

```
jwt.verify(token, obtenerSecreto(),  
  { algorithms: [ALGORITMO] });
```

Sin esa lista, el propio token podría decir en su cabecera con qué algoritmo hay que revisarlo. El ataque clásico es mandar `alg: "none"`, o sea un token sin firma, esperando que el servidor lo acepte. Con la lista, el que decide el algoritmo es el servidor, no el token. Es la misma idea de lista blanca del Laboratorio 8, aplicada ahora a la verificación.

El secreto se lee con `obtenerSecreto()`, que lanza un error si `JWT_SECRET` no existe. Así la API nunca firma con un secreto vacío.

```
// =====  
// Generar token  
// El id va en el claim estándar sub; nunca se incluye la contraseña  
// =====  
// El payload viaja codificado en base64, no cifrado: cualquiera que tenga el  
// token puede leerlo. La firma impide modificarlo, no lo oculta.  
const generarToken = (usuario) =>  
  jwt.sign(  
    { email: usuario.email, rol: usuario.rol },  
    obtenerSecreto(),  
    {  
      algorithm: ALGORITMO,  
      expiresIn: process.env.JWT_EXPIRES_IN || EXPIRACION_POR_DEFECTO,  
      subject: String(usuario.id)  
    }  
  );  
  
// =====  
// Verificar token  
// Solo se acepta HS256: así un token no puede imponer su propio algoritmo  
// =====  
const verificarToken = (token) =>  
  jwt.verify(token, obtenerSecreto(), { algorithms: [ALGORITMO] });
```

Figura 3. `src/utills/jwt.util.js`: `generarToken` firma `email` y `rol` con HS256 y el id en `sub`; `verificarToken` solo acepta HS256.

5. El middleware autenticarJWT

En `src/middlewares/auth.middleware.js` creamos `autenticarJWT`, el que revisa la manilla en cada ruta protegida. Hace cuatro cosas en orden:

- Lee la cabecera `Authorization`. Si no viene, responde 401 "Token de autenticación requerido".
- La parte por el espacio en `[esquema, token]`. Si el esquema no es `Bearer` o falta el token, responde 401 "Formato de token inválido".
- Llama a `verificarToken`. Si la firma es buena, guarda en `req.usuario` el id (sacado de `sub`), el email y el rol, y deja pasar la petición con `next()`.
- Si la verificación falla, responde 401 con un mensaje que distingue el token vencido del token inválido.

```
// src/middlewares/auth.middleware.js
const autenticarJWT = (req, res, next) => {
  const cabecera = req.get("Authorization");

  if (!cabecera) {
    return res
      .status(401)
      .json({ mensaje: "Token de autenticación requerido" });
  }

  const [esquema, token] = cabecera.split(" ");

  if (esquema !== "Bearer" || !token) {
    return res.status(401).json({ mensaje: "Formato de token inválido" });
  }

  try {
    const carga = verificarToken(token);

    req.usuario = {
      id: Number(carga.sub),
      email: carga.email,
      rol: carga.rol
    };

    next();
  } catch (error) {
    if (!ERRORES_DE_TOKEN.includes(error.name)) return next(error);

    if (error.name === "TokenExpiredError") {
      return res.status(401).json({ mensaje: "Token expirado" });
    }

    res.status(401).json({ mensaje: "Token inválido" });
  }
};
```

Figura 4. `autenticarJWT` en `src/middlewares/auth.middleware.js`, con el `catch` que separa los errores del token de cualquier otro error.

Una mejora en el catch

La primera versión del `catch` respondía 401 "Token inválido" ante cualquier error. El problema es que si fallaba algo que no tiene nada que ver con el token, por ejemplo que

faltara `JWT_SECRET`, el cliente recibía "Token inválido" y nosotros nunca nos enterábamos de que el servidor estaba mal configurado. Por eso agregamos esta línea:

```
if (!ERRORES_DE_TOKEN.includes(error.name))  
  return next(error);
```

Ahora solo los errores propios del token (`JsonWebTokenError`, `TokenExpiredError`, `NotBeforeError`) se convierten en 401. Cualquier otro sigue su camino hasta el manejador de errores de la aplicación, que responde 500 sin mostrar detalles internos.

Situación	Respuesta
No llega la cabecera Authorization	401 "Token de autenticación requerido"
La cabecera no es <code>Bearer <token></code>	401 "Formato de token inválido"
El token venció	401 "Token expirado"
Firma mala, token alterado u otro algoritmo	401 "Token inválido"
Error que no es del token	Pasa al manejador global (500)
Token correcto	Sigue a la ruta con <code>req.usuario</code>

6. El login ahora entrega el token

En `auth.controller.js`, cuando el login es correcto, el controller llama a `generarToken` y agrega el token a la respuesta. El resto quedó igual: si las credenciales fallan, sigue respondiendo 401 con el mensaje genérico "Credenciales inválidas". En Swagger documentamos la nueva respuesta con el schema `RespuestaLogin`.

Para la prueba registramos Asistente Teatro (asistente@teatro.com) y luego iniciamos sesión en Postman, con la API Key de Postman Laboratorio en la cabecera **X-API-Key**. La respuesta fue **200 OK** con el usuario público y el token:

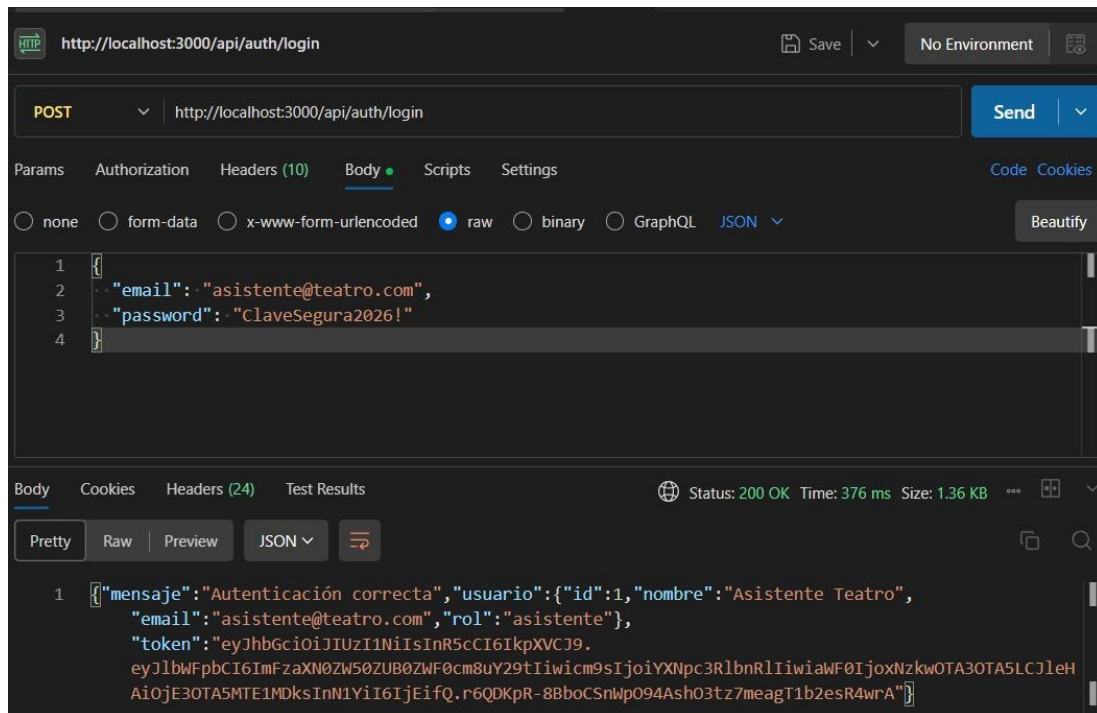


Figura 5. POST /api/auth/login con credenciales correctas: 200 OK con el usuario y el token JWT.

El token de la captura es de una sesión de prueba ya vencida, por eso no hay problema en mostrarlo. Las tres partes separadas por puntos se ven a simple vista: la primera empieza por `eyJhbGci`, que es `{"alg"` en base64.

7. La ruta /perfil y los dos candados

Para probar el token creamos `GET /api/auth/perfil`. Esta ruta pasa por dos controles seguidos: primero la API Key, que ya protegía todo `/api` desde el Laboratorio 6, y después `autenticarJWT`. Cada uno responde una pregunta distinta y deja su resultado en un lugar distinto de la petición:

Control	Pregunta que responde	Dónde deja el resultado
API Key (X-API-Key)	¿Qué aplicación hace la petición?	<code>req.clienteApi</code>
JWT (Authorization: Bearer)	¿Qué persona hace la petición?	<code>req.usuario</code>

Swagger: los dos candados a la vez

En `swagger.js` agregamos el esquema de seguridad `BearerAuth` junto al `ApiKeyAuth` que ya teníamos. En la ruta `/perfil` los declaramos dentro de un mismo objeto:

```
security: [{ "ApiKeyAuth": [], "BearerAuth": [] }]
```

En OpenAPI eso significa "los dos" (Y). Si los hubiéramos puesto como dos objetos separados en la lista, significaría "cualquiera de los dos" (O), y la documentación diría algo que el servidor no hace. En el botón Authorize de Swagger UI ahora aparecen ambos:

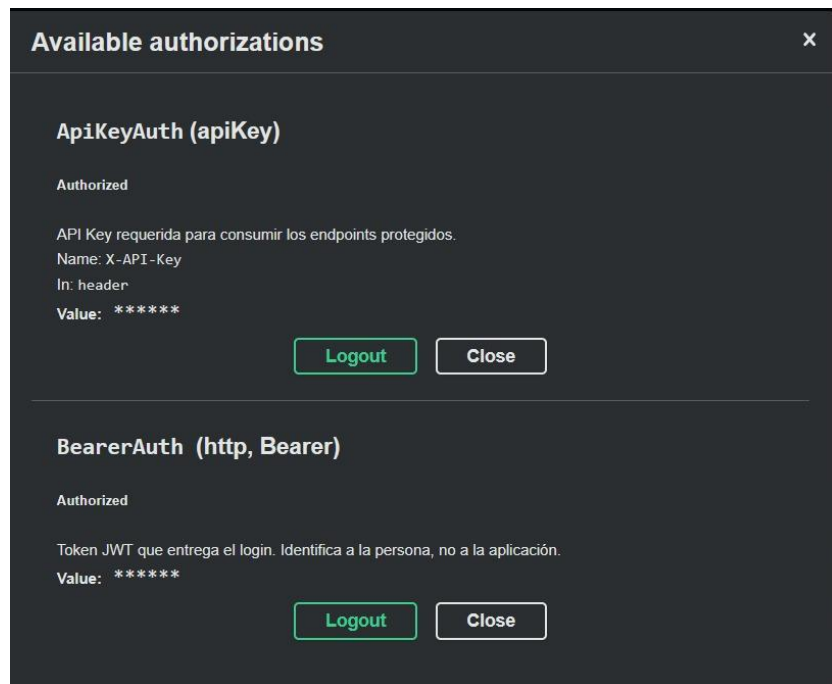


Figura 6. Authorize en Swagger UI con `ApiKeyAuth` y `BearerAuth` autorizados. Los valores se muestran ocultos.

Perfil con token válido

En Postman pusimos el token en la pestaña Authorization, tipo Bearer Token, y llamamos a `/perfil`. La respuesta `200 OK` muestra los dos candados funcionando juntos: `usuario` viene del token y `clienteApi` viene de la API Key.

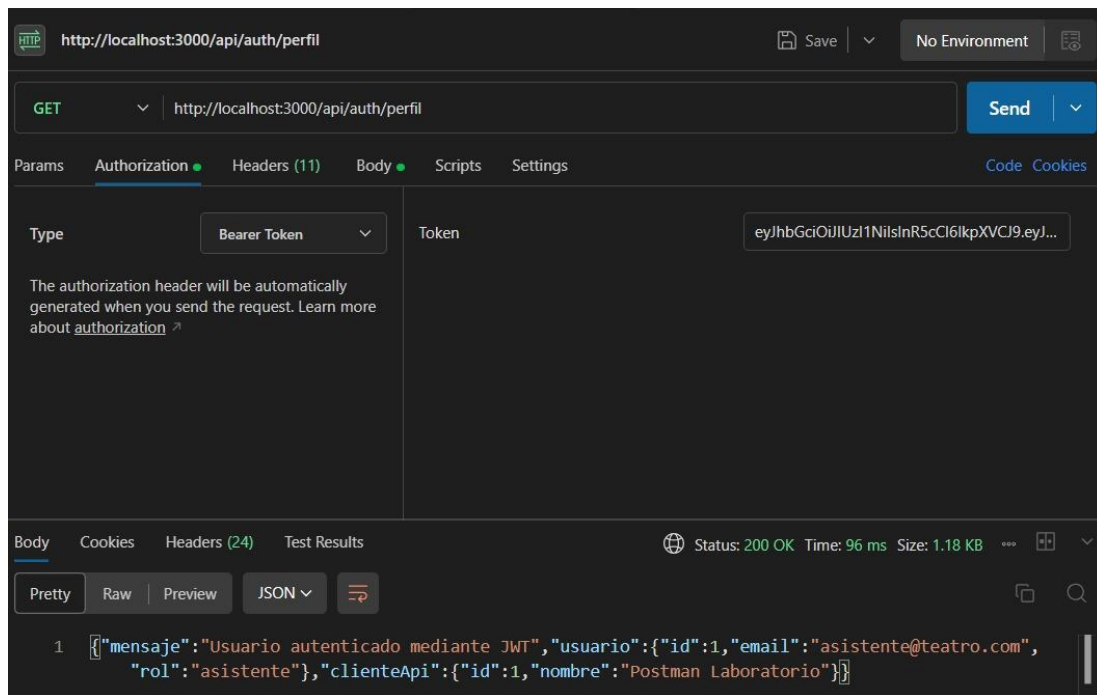


Figura 7. GET /api/auth/perfil con token válido: 200 OK con el usuario del token y el cliente de la API Key (Postman Laboratorio).

La respuesta no trae el nombre del usuario: `/perfil` devuelve lo que viene dentro del token (id, email y rol), no consulta la lista de usuarios. Eso tiene una consecuencia que explicamos en la sección 14.

8. Atacando nuestra propia API

Sin token

Con la misma petición pero con Authorization en No Auth, la API Key sigue llegando y aun así la ruta responde 401. Tener la aplicación autorizada no alcanza: falta saber quién es la persona.

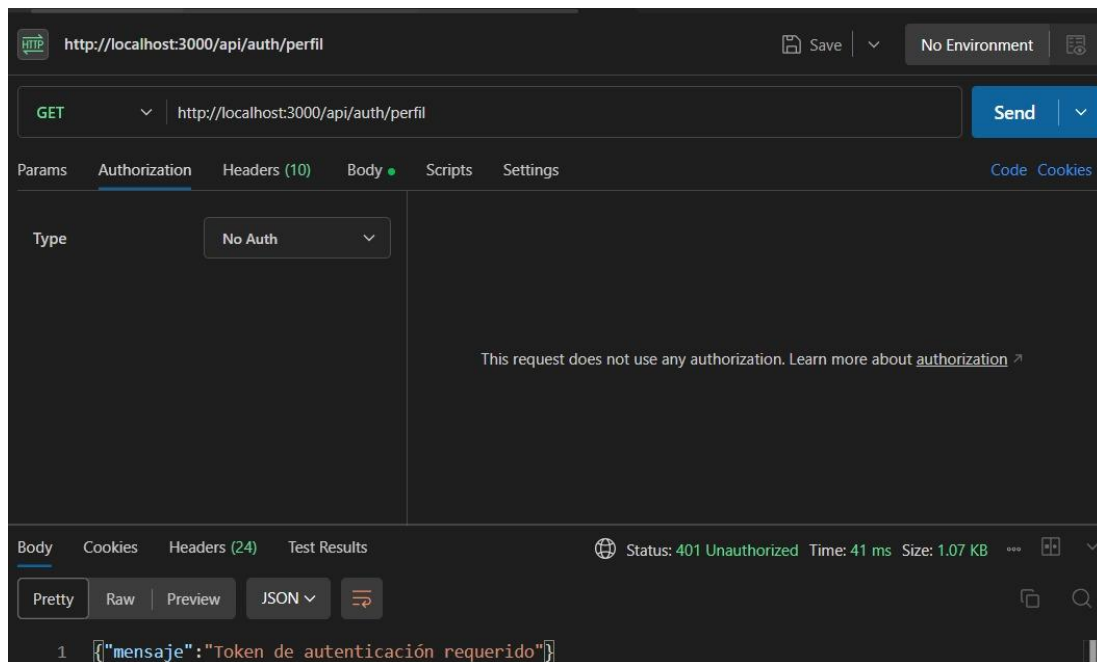


Figura 8. GET /api/auth/perfil sin token: 401 "Token de autenticación requerido".

Token alterado

Después tomamos el token válido y le cambiamos un carácter. Para la librería ya no coincide la firma con el contenido, así que lo rechaza:

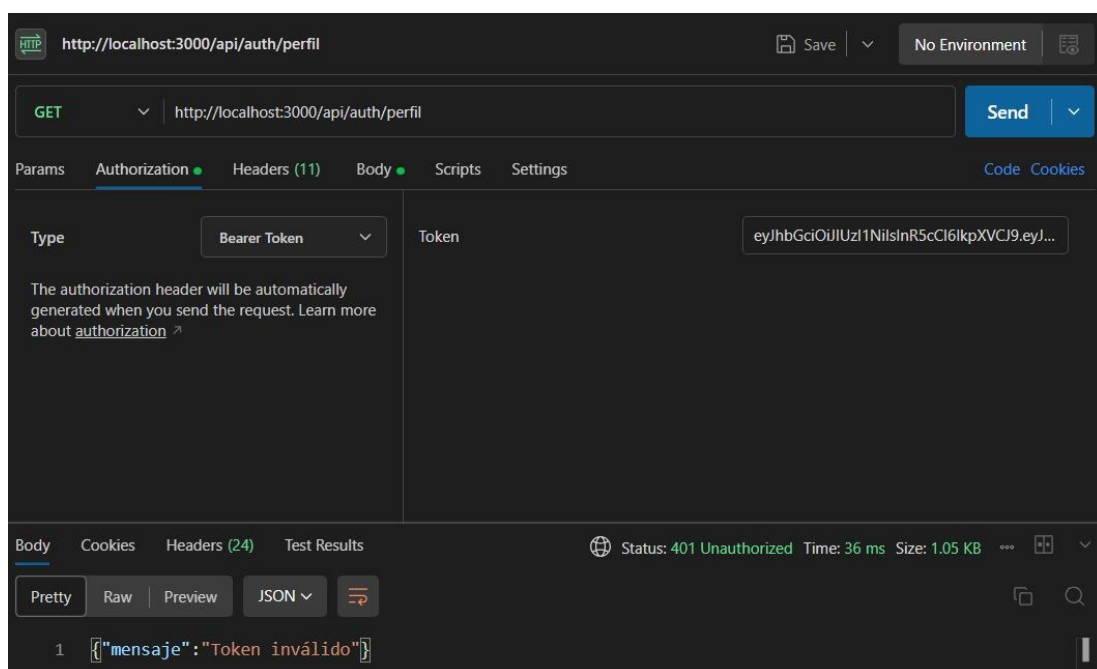


Figura 9. GET /api/auth/perfil con un token modificado: 401 "Token inválido".

Otras pruebas

Además de las capturas, probamos otros casos que un atacante intentaría. Todos dieron el resultado esperado:

Intento	Resultado
Token con alg : "none" y sin firma	401 "Token inválido"
Token firmado con otro secreto	401 "Token inválido"
Cambiar el payload a rol : "administrador" sin volver a firmar	401 "Token inválido"
Token válido pero sin la cabecera X-API-Key	401 de la API Key, nunca llega al JWT
Mismo token con la API Key de Taquilla	200, con clienteApi Taquilla: el token no está atado a una aplicación

La tercera fila es la más importante: aunque el payload se puede leer, no se puede cambiar. Si alguien edita su rol a administrador, la firma deja de coincidir y el token no sirve.

9. Leer no es verificar: `jwt.decode`

Para comprobar lo de la sección 3 usamos `jwt.decode` desde la terminal, con el token del login. `decode` no usa el secreto ni revisa la firma: solo lee el base64. Aun así, nos mostró todo el payload:

```
PS C:\Users\Juan_Camilo\OneDrive\Documentos\API_TEATRO> node -e "console.log(require('jsonwebtoken').decode('eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFrpbC11bmFzaXQwZm50ZU80ZmF0cm8uY291c2lucm91IjoieWxpbnRlbnR1IiwiaWF0IjoxNzkwOTA3OTAsIChhOiJlE30TASMTc0S2lnInR1IiwiaWF0IjE1fQ.r6QDkPr-8BbcSnpqO94A5ht3t7meagT1b2es4RwA'))")"
```

```
{
  email: 'asistente@teatro.com',
  rol: 'asistente',
  lat: 1706007909,
  exp: 1706911509,
  sub: '1'
}
```

```
PS C:\Users\Juan_Camilo\OneDrive\Documentos\API_TEATRO>
```

Figura 10. `jwt.decode` lee el payload sin conocer el secreto: `email`, `rol`, `iat`, `exp` y `sub`.

Ahí están los claims que pusimos: `email`, `rol` y `sub: '1'`. Si restamos `exp - iat`, nos da `1790911509 - 1790907909 = 3600` segundos, o sea exactamente la hora de `JWT EXPIRES IN=1h`.

La lección es doble. Primero, cualquiera con el token puede leer lo que lleva, por eso no metemos datos sensibles. Segundo, en el servidor nunca se debe usar `decode` para autenticar: leer el token no prueba que sea auténtico. Por eso el middleware usa `verify`.

10. El token vence

Para no esperar una hora, probamos la expiración con un token de 20 segundos. El `.env` no se toca: como `dotenv` no reemplaza las variables que ya existen en el sistema, basta con definirla en la terminal antes de arrancar el servidor:

```
$env:JWT_EXPIRES_IN="20s"; node src/app.js
```

Volvimos a registrar el usuario (los usuarios viven en memoria y se borran al reiniciar), hicimos login para recibir un token nuevo de 20 segundos, esperamos unos 25 y llamamos a `/perfil`:

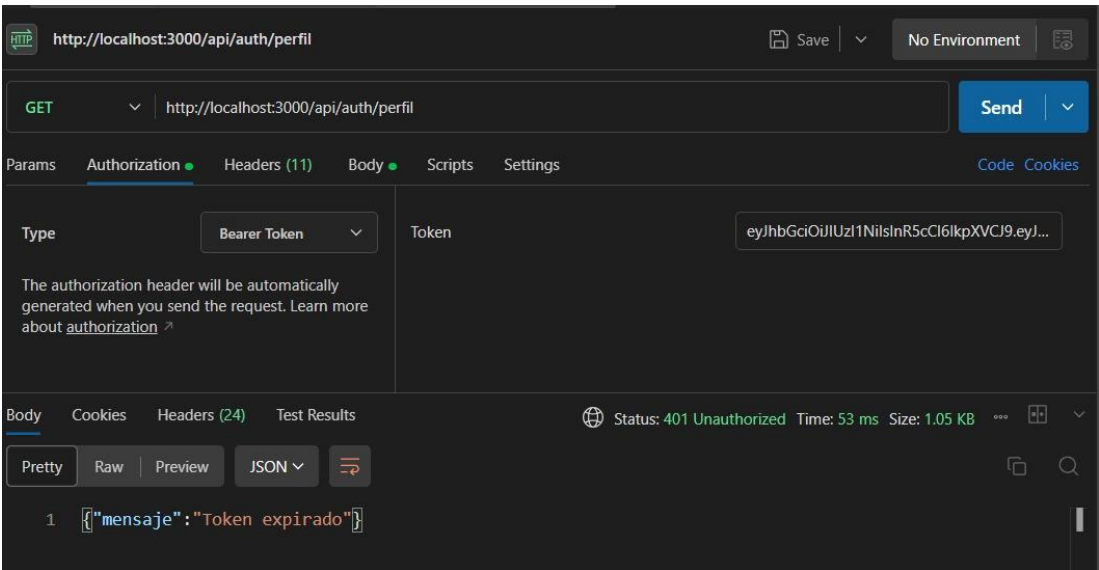


Figura 11. GET `/api/auth/perfil` con un token vencido: 401 "Token expirado".

Gracias al `catch` de la sección 5, el mensaje dice exactamente qué pasó, y el cliente sabe que tiene que volver a iniciar sesión. Al terminar quitamos la variable con `Remove-Item Env: JWT_EXPIRES_IN` para volver a la hora normal.

En el primer intento esta prueba nos dio 200: estábamos reusando un token viejo de una hora y teníamos dos procesos de Node abiertos. Lo resolvimos cerrando todos los procesos de Node y pidiendo un token nuevo después de arrancar con los 20 segundos.

11. API Key, JWT y rol: cada uno en lo suyo

Con este laboratorio la API ya tiene tres piezas de identidad, y es fácil confundirlas:

Pieza	Identifica a	Responde	Estado actual
API Key	La aplicación cliente	¿Esta app puede usar la API?	Activa desde el Lab 6
JWT	La persona	¿Quién inició sesión?	Activo en <code>/perfil</code>
Rol	Lo que la persona puede hacer	¿Tiene permiso para esto?	Viaja en el token, todavía no restringe

La API Key no reemplaza al JWT ni al revés: una app autorizada puede ser usada por muchas personas, y una misma persona puede entrar desde varias apps, como vimos con la prueba de Taquilla.

12. Checkpoint del laboratorio

Prueba	Esperado	Obtenido	Estado
Login correcto	200 + token	200 + token	OK
Perfil con token válido	200 + usuario y clienteApi	200 + usuario y clienteApi	OK
Perfil sin token	401	401 "Token de autenticación requerido"	OK
Perfil con token alterado	401	401 "Token inválido"	OK
Perfil con token vencido	401	401 "Token expirado"	OK
Token con alg none	401	401 "Token inválido"	OK
Payload leído con decode	Se lee sin secreto	email, rol, sub, iat, exp	OK
Contraseña dentro del token	No aparece	No aparece	OK

13. Verificación con Semgrep y npm audit

Al terminar volvimos a pasar las herramientas SAST y SCA:

Herramienta	Lab 8	Lab 9
Semgrep: archivos escaneados (73 reglas)	41	43
Semgrep: hallazgos	0	0
npm audit: paquetes	146	159
npm audit: vulnerabilidades	0	0

Los archivos pasaron de 41 a 43 por los dos nuevos: `jwt.util.js` y `auth.middleware.js`. Los paquetes subieron de 146 a 159 porque `jsonwebtoken` trae 13 dependencias propias. Ninguno tiene vulnerabilidades conocidas. Con la nueva ruta `/perfil`, la API pasó de 41 endpoints en 21 rutas a 42 endpoints en 22 rutas.

En el computador donde corrimos el análisis, el Control inteligente de aplicaciones de Windows bloquea `semgrep.exe`. No lo desactivamos; en su lugar ejecutamos el mismo Semgrep desde Python, con las mismas reglas (`p/javascript`, `p/nodejs` y `p/owasp-top-ten`):

```
python -c "from semgrep.console_scripts.pysemgrep
import main; main()" --config p/javascript
--config p/nodejs --config p/owasp-top-ten src/
```

```
Scan Summary
✓ Scan completed successfully.
• Findings: 0 (0 blocking)
• Rules run: 73
• Targets scanned: 43
• Parsed lines: ~100.0%
• Scan was limited to files tracked by git
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 73 rules on 43 files: 0 findings.
💎 Missed out on 2254 pro rules since you aren't logged in!
⚡ Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.
(need more rules? `semgrep login` for additional free Semgrep Registry rules)

📦 A new version of Semgrep is available. See https://semgrep.dev/docs/upgrading

🌟 If Semgrep missed a finding, please send us feedback to let us know!
See https://semgrep.dev/docs/reporting-false-negatives/
PS C:\Users\Juan Camilo\OneDrive\Documentos\API_TEATRO>
```

Figura 12. Resumen de Semgrep tras el Laboratorio 9: 73 reglas sobre 43 archivos y 0 hallazgos.

Las salidas completas quedaron guardadas en `docs/seguridad/sast-semgrep-lab9.txt` y `docs/seguridad/sca-npm-audit-lab9.txt`.

14. Lo que este laboratorio no resuelve

- **El rol todavía no restringe nada.** Viaja en el token y llega a `req.usuario.rol`, pero ninguna ruta lo revisa. Un asistente con token puede hacer lo mismo que un administrador. La autorización por rol es el siguiente paso.
- **No hay forma de revocar un token.** Si se lo roban, sirve hasta que venza. No hay logout real ni lista negra de tokens.
- **El token no se entera de los cambios.** Si a un usuario le cambian el rol o lo desactivan, su token viejo sigue diciendo lo de antes hasta que expire.
- **Un token sobrevive aunque el usuario ya no exista.** Lo vimos probando: después de reiniciar el servidor, los usuarios en memoria se borran, pero un token anterior que no había vencido seguía funcionando en `/perfil`, porque la ruta confía en la firma y no consulta si el usuario existe.
- **JWT_SECRET se valida al usarse, no al arrancar.** Si falta, el servidor inicia normal y el error aparece en el primer login. Sería mejor que no arrancara.

15. Repositorio

<https://github.com/JuanK2550/API-TEATRO>

La autenticación con JWT está en el commit `0548c87`, y el informe del laboratorio con las salidas de Semgrep y npm audit en `4249caa`.